

Workflow Automation Platform

Project Documentation — n8n-style Build

Complete reference for tech stack, architecture, system design, and build order for a self-built visual workflow automation platform.

React + NestJS + TypeScript | DAG-based Execution Engine

Prepared as a personal project reference • July 2026

1. Project Overview

1.1 What This Platform Is

A visual workflow-automation tool, similar in concept to n8n — users connect "nodes" (triggers, actions, conditions) on a drag-drop canvas to automate tasks: syncing data between apps, responding to webhooks, running scheduled jobs, and chaining API calls together without writing glue code by hand.

1.2 What n8n Actually Is, Architecturally

Under the hood, n8n's real architecture is frontend-backend separated: a visual editor converts flows into JSON, an execution engine loads that JSON and runs it node-by-node, and a plugin-based node system handles the individual integrations. This is the same shape we're building.

Component	Responsibility
Visual Editor (Frontend)	Drag-drop UI; converts the visual flow into a JSON graph definition
Workflow Storage	Database storing workflow definitions as JSON/graph structures
Execution Engine	Reads the JSON graph and executes nodes in the correct order
Node System	Pluggable units — each node does one thing (API call, transform, condition, etc.)
Trigger System	Webhooks, cron schedulers, event listeners that start a workflow run
Queue / Worker System	Handles async execution at scale, retries, and concurrency
Credential Management	Securely stores API keys/tokens per integration, scoped per workflow/user

Reality check: This is a serious multi-month systems project, not a weekend build. Even mature workflow tools can struggle with very high concurrency at large scale — plan to build in small, working slices (see Section 5) rather than attempting the whole system at once.

2. Tech Stack Decision

2.1 Final Stack

Layer	Choice	Reasoning
Frontend	React + React Flow	React Flow is the standard library for building drag-drop node/graph canvases
Backend	NestJS (on Node.js runtime)	Structured, modular, decorator-based — maps naturally to "each node type is a plugin"; same language as the future SDK
Language	TypeScript everywhere	Shared types between frontend, backend, and SDK — no translation layer, no schema drift
Queue / Workers	Redis + BullMQ	Executes workflows asynchronously, handles retries and horizontal scale
Database	PostgreSQL	Stores workflow definitions (as JSON/graph), execution logs, credentials
Deployment	Docker → Kubernetes (later)	Standard path for horizontally scaling the execution/worker layer

2.2 Why NestJS Over Plain Express

- Built-in **dependency injection** — makes testing and swapping implementations easy as the platform grows
- **Module system** — each integration/node type becomes a self-contained, pluggable module
- **Guards, interceptors, pipes** — auth, logging, and validation patterns ship built-in instead of hand-rolled
- Structure scales with the project instead of turning into an unstructured pile of routes

2.3 Why This Stack Fits the Long-Term SDK Goal

Since the entire third-party integrations ecosystem (Slack, Stripe, Google, Notion SDKs) is overwhelmingly JS/TS-first, building the platform in the same language means the future public SDK shares types directly with the backend — no schema drift, no translation layer, and it stays cheap to maintain as more integrations get added.

2.4 Monorepo Structure (set this up from Day 1)

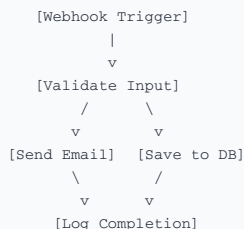
```
/apps
  /frontend      -> React app (canvas / node editor)
  /backend       -> NestJS app (API, auth, workflow storage)
  /worker        -> Execution engine (consumes queue, runs DAGs)
/packages
  /sdk           -> Public SDK (shared types with backend)
  /shared-types  -> TypeScript types used across frontend, backend, worker
```

Managed with **pnpm workspaces** or **Turborepo**. Setting this up early makes "turn this into an SDK later" cheap instead of a painful retrofit down the line.

3. System Design — Core Concepts

3.1 The Core Concept: DAG (Directed Acyclic Graph)

A workflow-automation platform is fundamentally a **graph execution system** wearing a visual UI. Every workflow built on the canvas is a **DAG** — a set of steps (nodes) connected by directional edges, with *no cycles* (a step can never loop back and create an infinite dependency on itself).



Key properties this gives as a system designer:

- **Directed** — data flows one way, from trigger to outcome.
- **Acyclic** — no infinite loops in the dependency chain (loops for repetition live inside a node, not as a graph cycle).
- **Each node** takes input from its parent(s), does one job, and passes output to its child(ren).
- **Parallel branches** (like the example above) can execute concurrently, then join back together.

3.2 Execution Flow (step by step)

1. User builds a workflow visually → frontend serializes it into JSON (nodes + edges)
2. Backend (NestJS) validates and stores this JSON as the workflow definition in PostgreSQL
3. A trigger fires (webhook hit, cron schedule, manual run)
4. Backend pushes an "execute this workflow" job onto the queue (Redis/BullMQ)
5. A worker process picks up the job, loads the DAG, and walks it node-by-node
6. Each node's output becomes the next node's input; failures are logged with retry logic
7. Execution history/logs are stored for traceability and debugging

Why separate the API server and the Worker? The API server (NestJS) should stay fast and only handle requests/storage. Actual execution (which can be slow, retry-heavy, or long-running) happens in separate worker processes. This separation is what allows horizontal scaling — more workers can be added under load without touching the API layer at all.

3.3 Known Limitations to Design Around

- Workflow-based systems can struggle with very high concurrency compared to purpose-built distributed backends — plan for this early if scale matters.
- Complex business logic (deep nested conditionals, heavy algorithms) is better handled in a dedicated code node, not spread across many small visual nodes.
- Data consistency across multiple services/steps is harder to guarantee than a single database transaction — design compensating actions (rollback/cleanup steps) for critical workflows.
- Always add retry logic and dead-letter queues for webhook handling — networks fail, webhooks get missed.

4. Node System Design

4.1 What a "Node" Actually Is

Each node is a self-contained unit with a defined **input schema**, **output schema**, and an **execute function**. Two broad categories:

Type	Purpose	Examples
Trigger Nodes	Start a workflow run	Webhook, Schedule/Cron, Manual Trigger
Action Nodes	Do something with data	HTTP Request, Send Email, Database Write, Transform/Map Data, Conditional (If/Switch)

4.2 Minimal Node Interface (conceptual)

```
interface NodeDefinition {
  type: string;           // e.g. "http_request", "send_email"
  inputSchema: JSONSchema; // what this node expects as input
  outputSchema: JSONSchema; // what this node produces
  execute(input, credentials): Promise<output>;
}
```

4.3 Credential Handling

- Credentials (API keys, OAuth tokens) are stored encrypted, separate from workflow definitions
- Each node references a credential by ID, never embeds it directly in the workflow JSON
- Scope credentials per user/workflow to avoid one leaked workflow exposing everything

5. Suggested Build Order

Don't build the visual UI and the execution engine at the same time — this is where most solo builders stall out. Build in this order instead:

1. **Slice 1 — Core Engine (no UI):** One webhook trigger → one execution engine → 3–4 hardcoded node types. Workflows defined directly as JSON files, no visual editor yet.
2. **Slice 2 — Async Execution:** Add the queue system (Redis + BullMQ) so execution is async and retryable instead of blocking.
3. **Slice 3 — Persistence:** Add PostgreSQL to store workflow definitions and execution logs/history.
4. **Slice 4 — Visual Canvas:** Build the React + React Flow drag-drop editor on top of the now-working engine.
5. **Slice 5 — Auth & Multi-tenancy:** Add credential management, user auth, and workflow ownership/permissions.
6. **Slice 6 — SDK Extraction:** Pull out the shared types and publish the first internal SDK package.

Why this order matters: Get a working, testable engine first — even if it's ugly and JSON-only — then layer the visual experience on top of something that already works. Building the UI before the engine works means debugging two unstable systems at once.

Slice	Status
Slice 1 - Core Engine (no UI)	🚧 Not started
Slice 2 - Async Execution (Queue)	🚧 Not started
Slice 3 - Persistence (PostgreSQL)	🚧 Not started
Slice 4 - Visual Canvas (React Flow)	🚧 Not started
Slice 5 - Auth & Multi-tenancy	🚧 Not started
Slice 6 - SDK Extraction	🚧 Not started

This document is the master reference for the n8n-style automation platform project. Update the slice status table as each stage is completed. Paste this into any new chat to restore full project context.